Experiment No. 12

End-to-End Communication at the Transport Layer Using TCP and UDP Socket Programming

Aim

To implement end-to-end communication at the transport layer using socket programming in Python. To develop TCP and UDP echo client-server programs and a chat application, and to understand how data is transmitted between client and server processes using transport-layer protocols.

Procedure

Part A: TCP Echo Client-Server Program

1. **Open Visual Studio Code and create a new folder named CN_Experiment_12. Create two Python files named tcp_server.py and tcp_client.py inside the folder.**
2. **In tcp_server.py, import the socket module and create a TCP socket using socket.socket(socket.AF_INET, socket.SOCK_STREAM). The AF_INET parameter specifies IPv4 addressing, while SOCK_STREAM specifies TCP communication.**
3. **Bind the server socket to the loopback IP address 127.0.0.1 and port number 5000. Use the listen() function to make the server wait for incoming client connections.**
4. **Use the accept() function to accept a connection from the client. When a client connects, the server receives the data using recv(). It sends the received message back to the client using sendall(), thereby implementing an echo server.**
5. **In tcp_client.py, create a TCP socket and connect it to 127.0.0.1 on port 5000. Read a message from the user, convert it into bytes, and transmit it to the server using sendall().**
6. **Receive the response from the server using recv() and display it on the screen. Run the server program first in one VS Code terminal and then run the client program in another terminal.**
7. **Enter a sample message in the client terminal. Verify that the server returns the same message. Repeat the test with different messages and terminate the client using the exit condition implemented in the program.**

Part B: UDP Echo Client-Server Program

8. **Create two additional files named udp_server.py and udp_client.py. Import the socket module in both programs.**

9. **In udp_server.py, create a UDP socket using socket.socket(socket.AF_INET, socket.SOCK_DGRAM). Bind the socket to 127.0.0.1 and port 5001. Unlike TCP, UDP does not require the server to establish a connection before receiving messages.**
10. **Use recvfrom() to receive a message and the client's address. Send the received message back to the client using sendto(). Continue receiving messages until the termination condition is reached.**
11. **In udp_client.py, create a UDP socket and specify the server address as 127.0.0.1 with port 5001. Read a message from the user and transmit it using sendto().**
12. **Receive the server's reply using recvfrom() and display the response. Run the UDP server in one terminal and the UDP client in another terminal. Send multiple messages and verify that each message is echoed by the server.**
13. **Observe the difference between TCP and UDP communication. TCP establishes a connection and provides reliable, ordered delivery of data. UDP sends independent datagrams without establishing a connection and does not guarantee delivery or ordering.**

Part C: Client-Server Chat Application

14. **Create two more files named chat_server.py and chat_client.py. These programs use TCP sockets to provide message exchange between a client and a server.**
15. **In chat_server.py, create a TCP socket, bind it to 127.0.0.1 and port 5002, and call listen() to wait for a connection. Accept the client connection using accept().**
16. **After accepting the connection, receive a message from the client and display it on the server terminal. Read a reply from the server operator and send it to the client. Repeat this process to allow a conversation between the two programs.**
17. **In chat_client.py, create a TCP socket and connect to the server using the same IP address and port number. Allow the client to enter messages, send them to the server, and receive replies from the server.**
18. **Run chat_server.py first and then execute chat_client.py in a separate terminal. Exchange several messages between the client and server. Use the termination condition implemented in the programs to end the conversation and close the connection.**
19. **Test all three applications with different input messages. Observe the messages displayed in the client and server terminals and confirm that the received messages match the transmitted messages.**

20. Close the client and server programs after testing. Compare the behaviour of TCP echo, UDP echo, and TCP chat communication to understand the role of transport-layer protocols in end-to-end data exchange.

tcp_server.py

```
import socket
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(("127.0.0.1", 5000))
server.listen(1)
print("TCP Echo Server started...")
conn, addr = server.accept()
print("Connected:", addr)

while True:
    data = conn.recv(1024)
    if not data:
        break
    print("Client:", data.decode())
    conn.send(data)

conn.close()
server.close()
```

tcp_client.py

```
import socket
```

```
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect(("127.0.0.1", 5000))

print("Connected to TCP Echo Server")

while True:
    msg = input("Enter message: ")
    if msg.lower() == "exit":
        break

    client.send(msg.encode())
    data = client.recv(1024)

    print("Server:", data.decode())

client.close()
```

udp_server.py

016-Akshaya K

```
import socket

server = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
server.bind(("127.0.0.1", 5001))

print("UDP Echo Server started...")

while True:
    data, addr = server.recvfrom(1024)
    print("Client:", data.decode())

    if data.decode().lower() == "exit":
        break

    server.sendto(data, addr)

server.close()
```

udp_client.py

016-Akshaya K

```
import socket

client = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

print("Connected to UDP Echo Server")

while True:
    msg = input("Enter message: ")

    client.sendto(msg.encode(), ("127.0.0.1", 5001))

    if msg.lower() == "exit":
        break

    data, addr = client.recvfrom(1024)
    print("Server:", data.decode())

client.close()
```

016-Akshaya K

chat_server.py

```
import socket
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(("127.0.0.1", 5002))
server.listen(1)
print("Chat Server started...")
conn, addr = server.accept()
print("Connected:", addr)
while True:
    msg = conn.recv(1024).decode()

    if msg.lower() == "exit":
        break

    print("Client:", msg)

    reply = input("You: ")
    conn.send(reply.encode())

    if reply.lower() == "exit":
        break

conn.close()
server.close()
```

chat_client.py

```
import socket
```

016-Akshaya K

```
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect(("127.0.0.1", 5002))

print("Connected to Chat Server")

while True:
    msg = input("You: ")
    client.send(msg.encode())

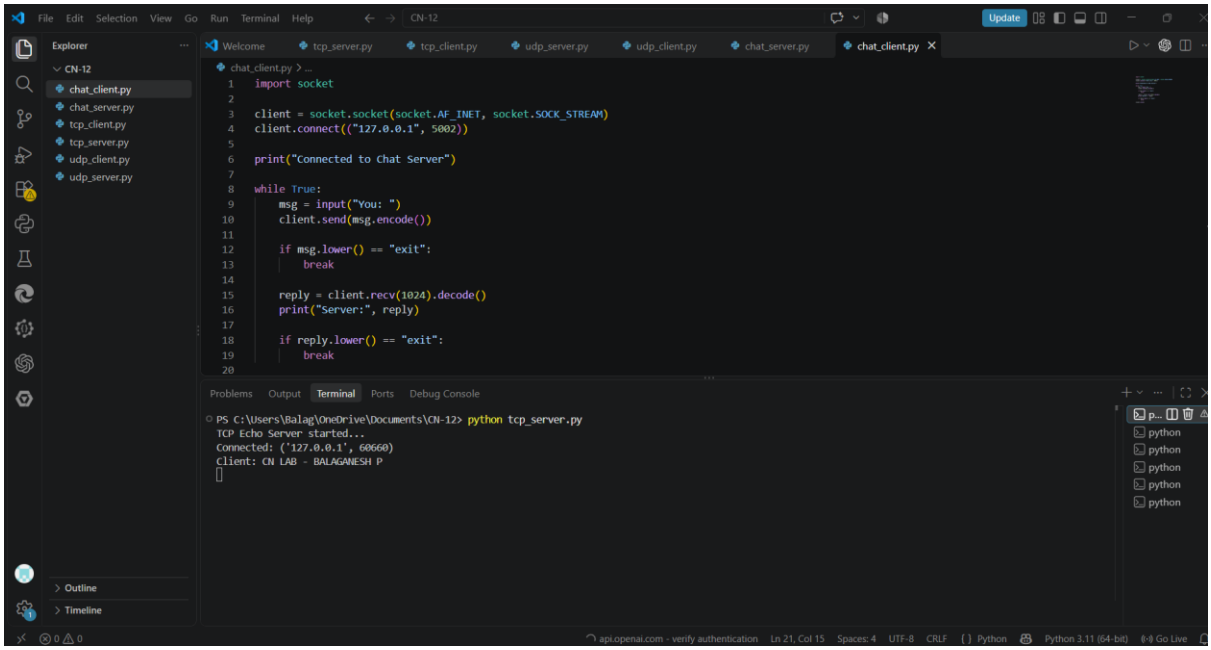
    if msg.lower() == "exit":
        break

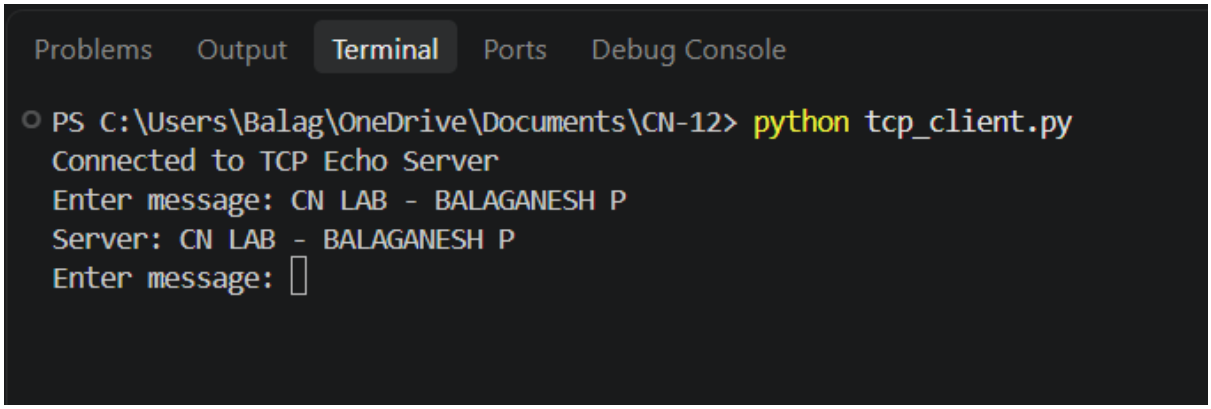
    reply = client.recv(1024).decode()
    print("Server:", reply)

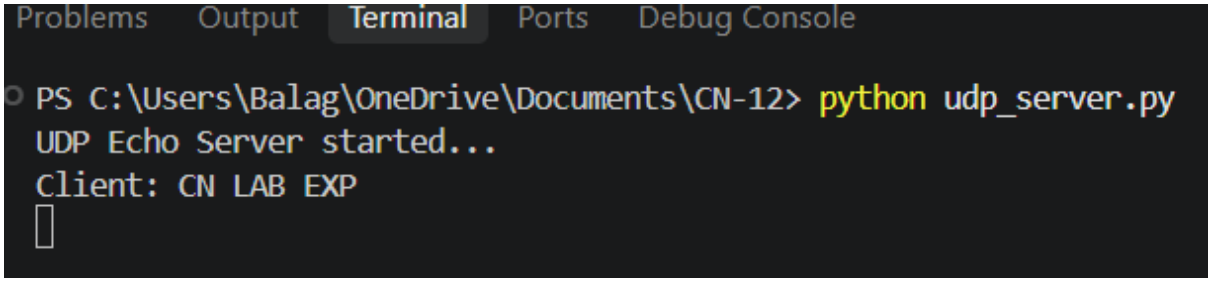
    if reply.lower() == "exit":
        break

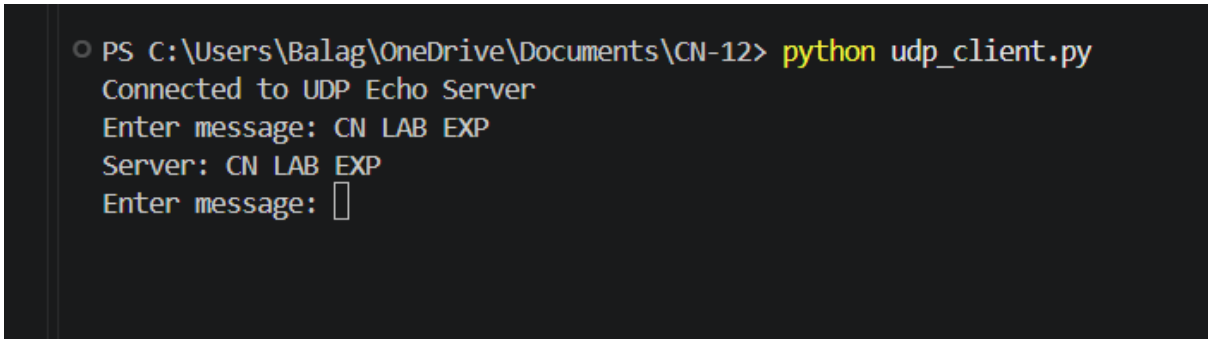
client.close()
```

016-Akshaya K









```
PS C:\Users\Balag\OneDrive\Documents\CN-12> python chat_server.py
Chat Server started...
Connected: ('127.0.0.1', 58511)
Client: HII
You: GOOD MORNING
Client: WELCOME
You:
```

```
Problems  Output  Terminal  Ports  Debug Console

PS C:\Users\Balag\OneDrive\Documents\CN-12> python chat_client.py
Connected to Chat Server
You: HII
Server: GOOD MORNING
You: WELCOME
```

Result

Thus, end-to-end communication at the transport layer was implemented using Python socket programming. TCP and UDP echo client-server programs were developed and tested, and a TCP-based chat application was implemented. The transmitted messages and server responses were verified successfully. The experiment demonstrated the differences between TCP's connection-oriented, reliable communication and UDP's connectionless datagram-based communication.